Pointy - Current State Report

Prepared: August 2026
Status: development snapshot; dev work paused at the user's request

1. Executive summary

Pointy is a Windows-first Tauri desktop assistant that answers questions about the user's current application, speaks the answer, and points at a UI control. It also contains a guided walkthrough mode for multi-step tasks, accessibility-based pointer refinement, local misclick prevention, usage tracking, and latency instrumentation.

The central design rule is:

> **The AI suggests what to do. The operating system and local application state decide whether the user actually did it.**

Pointy is not a continuous video agent. It does not stream screen frames while idle. A screenshot and model request are made only after an explicit user query, or after a locally verified completion of an active guided step.

This document describes what is currently implemented in the checkout. It does not claim that every planned macOS, wake-word, billing, or cloud product feature is complete.

2. Current architecture

Desktop shell

- Tauri 2 application.
- Rust backend in `src-tauri`.
- React and TypeScript overlay/frontend in `src`.
- Webview overlay is transparent, always-on-top, and can be made mouse-event-transparent.
- Rust owns OS access, screen capture, accessibility lookup, API calls, speech fallback, and local usage data.

Main components

- `lib.rs`: Tauri commands and application state.
- `nim.rs`: provider cascade, vision/text calls, streaming, STT, JSON parsing, model-output cleanup.
- `capture.rs`: window/monitor enumeration and screenshot capture.
- `overlay.rs`: full virtual-desktop overlay, capture conceal/show, click-through handling.
- `uia.rs`: Windows UI Automation lookup, raw rectangles, pointer centers, snapshots, confusion zones.
- `events.rs`: Windows UI Automation event subscriptions and debounce.
- `guide.rs`: guided walkthrough loop and local completion gate.
- `task_state.rs`: Trust Layer state machine and transition diagnostics.
- `misclick.rs`: local cursor velocity/dwell safety watcher.
- `tts.rs`: browser speech path plus Windows SAPI and cached warning audio.
- `usage.rs`: local usage tracking and local usage questions.
- `pointy.ts`, `overlay.tsx`, and overlay components: typed bridge and user interface.

3. Single-question flow

1. The user wakes Pointy with the configured hotkey and chooses a target application window.

2. The user speaks or types a question.
3. The overlay is concealed for capture. The selected window is focused.
4. Rust captures the selected window directly when possible. It falls back to monitor capture and crop if direct capture is unavailable.
5. The image is downscaled to a maximum long edge of 1280 pixels and encoded as JPEG.
6. Rust sends the question and screenshot to the first available provider in the configured cascade.
7. The model returns JSON containing an answer, optional advice, action, confidence, and an approximate target box.
8. When a target label is available, Windows UI Automation searches the selected window for the real control.
9. The model box is replaced with the accessibility rectangle when a match is found.
10. The frontend displays the answer and can show a high-contrast animated box and center dot.
11. The answer is spoken through Web Speech in the webview, with Windows SAPI as fallback when no browser voice is available.

Single-query mode does not start the guided loop unless the response is explicitly multi-step and is safe to verify locally.

4. Guided walkthrough logic

Guided mode is started only when all of the following are true:

- the model marks the task as multi-step;
- the first target exists;
- the target has a real accessibility dot;
- model confidence is at least 0.65;
- the action is one of click, type, select, toggle, submit, or open.

The first instruction is shown immediately from the single-question response. The backend then creates a local step contract containing:

- step number;
- action type;
- target label;
- confidence;
- current Trust Layer phase.

Trust Layer phases

- `waiting_for_action`: the user is being shown one instruction.
- `requesting_next`: a local completion signal was accepted and the next model call may begin.
- `recovery`: a long period of waiting produced a gentle check-in.
- `completed`: the model reported the task is complete.
- `stopped`: the guide is no longer active.

Completion gates

A generic focus change is never enough. A generic accessibility redraw is never enough.

The strongest path is a local mouse-button edge whose physical cursor position is inside the raw accessibility rectangle of the highlighted target. That signal is handled immediately, so Pointy does not wait for a browser redraw or structure event that may never arrive.

A secondary path handles a newly opened window or dialog. It takes an accessibility snapshot only after the event and accepts it only when the tree shows a meaningful change, such as a toggle change or a large structural change.

Only after one of these gates succeeds does Pointy capture a new screenshot and request the next instruction.

Check-ins and stopping

The guide waits patiently for up to 50 seconds before speaking a check-in. This is not a hard timeout. It continues waiting afterward. Stop is checked in short slices, and in-flight results are discarded after Stop so late events cannot re-open or speak the guide.

5. Windows accessibility event system

`events.rs` creates a dedicated COM STA thread and message loop. It registers:

- `UIA_AutomationFocusChangedEventId`;
- `UIA_StructureChangedEventId`;
- `UIA_Window_WindowOpenedEventId`.

Events are debounced in a 250 millisecond window. A burst of internal redraw events is collapsed into one callback. Each debounced event is logged as an `EVENT:` line. Pointy's own process ID is filtered so overlay hide/show operations do not look like user progress.

Idle behavior is intentionally quiet: event listeners remain registered, but no screenshot and no AI call occurs while the guide is waiting.

The current implementation is Windows-tested. macOS Accessibility observer work is not fully implemented or verified in this checkout.

6. Pointer positioning algorithm

The model is used to name a likely control. UI Automation is used to locate the actual control.

For the raw accessibility rectangle:

```
raw rectangle = (x, y, width, height)
dot_x = x + width / 2
dot_y = y + height / 2
```

The rectangle and center are kept in physical virtual-desktop pixels. The code also computes fractions of the complete virtual desktop for frontend rendering.

The overlay window is placed at the virtual desktop origin and sized to the union of all monitors. This supports:

- a monitor to the left of the primary monitor;
- negative virtual-screen coordinates;
- multiple monitors;
- high-DPI displays.

The code logs the raw rectangle, DPI scale, computed center, and overlay origin with a `POSITION:` line. The overlay is set to ignore cursor events except over the Pointy card, so the pointer layer does not block the real application click.

On Windows, UI Automation coordinates are consumed in the same physical screen coordinate space as `GetCursorPos`. The macOS Y-flip requirement is not yet implemented because the macOS observer path is not complete.

7. Local misclick prevention

Misclick prevention runs only while a guided step with a resolved target is active. It does not run while Pointy

is idle and it does not call a model or network service.

Setup

1. UI Automation walks the selected window's descendants.
2. Only interactive controls are considered: buttons, checkboxes, links, tabs, list items, sliders, edits, menu items, and similar controls.
3. Controls outside a 240-pixel confusion radius are ignored.
4. The target itself and heavy-overlap matches are excluded.
5. At most six nearby zones are retained.

Reactive decision

The watcher samples the OS cursor approximately every 8 milliseconds. For each zone it tracks:

- whether the cursor is inside;
- continuous dwell duration;
- approach speed;
- current speed;
- cooldown state.

A warning fires only when:

- the cursor newly entered the wrong zone;
- dwell is at least 200 milliseconds;
- current speed has fallen below 45 percent of approach speed, or both speeds are very low.

A fast pass-through or brief grazing movement does not warn. Moving onto the correct target cancels pending warnings. A zone has a 2.5 second cooldown and lingering does not repeatedly fire.

The warning is a pre-cached WAV generated once through Windows SAPI and played asynchronously with `PlaySoundW`. The reactive path does not create a process, call TTS live, call AI, or use the network. It emits `guide://warn` so the correct dot can briefly brighten.

Verification logs use this shape:

```
MISCLICK: zone=... dwell_ms=... approach_px_s=... current_px_s=... velocity_drop=... trigger_to_audio_ms=...
```

8. Latency pipeline

The guided cycle records these timestamps:

- `T0`: local completion event accepted;
- `T1`: screenshot captured;
- `T2`: streaming request sent;
- `T3`: first model content token received;
- `T4`: full response received;
- `T5`: target dot resolved and emitted;
- `T6`: first sentence begins speaking.

The log format is:

```
LATENCY: T0=... T1=... T2=... T3=... T4=... T5=... T6=... | event_to_screenshot=...ms screenshot_to_request=...ms | request_to_first_token=...ms generation_time=...ms | total=...ms
```

The active path uses:

- direct selected-window capture when supported;

- JPEG downscaling before the API request;
- one shared blocking `reqwest` client with connection pooling;
- SSE streaming for guided responses;
- first complete sentence speech before the full model response finishes.

Previously recorded development numbers were approximately 188 milliseconds for capture/downscale/encode and approximately 1.5 seconds for a model round trip. These are historical measurements, not a guarantee: free-provider rate limits and network conditions can add seconds.

9. AI and speech APIs

All provider keys are loaded by Rust from environment variables or local `.env` files. They are intentionally unprefixed. Do not use `VITE_` prefixes because Vite would expose those values to the webview bundle.

Vision and text cascade

Providers are tried in this order when a usable key exists:

1. Groq
- Endpoint: `https://api.groq.com/openai/v1/chat/completions`
- Key: `GROQ_API_KEY`
- Vision model: `qwen/qwen3.6-27b`
- Text model: `llama-3.3-70b-versatile`
- Requests disable reasoning where supported to reduce leaked thinking and latency.

2. Google Gemini
- Endpoint: `https://generativelanguage.googleapis.com/v1beta/openai/chat/completions`
- Keys: `GEMINI_API_KEY` or `GOOGLE_API_KEY`
- Models: `gemini-3.6-flash`, with `gemini-3.5-flash-lite` as a text fallback.

3. OpenRouter
- Endpoint: `https://openrouter.ai/api/v1/chat/completions`
- Keys: `OPEN_ROUTER_API_KEY` or `OPENROUTER_API_KEY`
- Free vision models: `google/gemma-4-26b-a4b-it:free`, `google/gemma-4-31b-it:free`, and `nvidia/nemotron-nano-12b-v2-vl:free`.
- Free text model: `google/gemma-4-26b-a4b-it:free`.

The OpenRouter Gemma model requested earlier is still present. OpenRouter is not the only configured provider: Groq and Gemini are attempted first when their keys exist. A provider that returns a rate limit, server error, or connection failure is placed in an 8-second cooldown to avoid hammering it.

Speech-to-text

Speech transcription uses OpenAI-compatible multipart audio endpoints:

1. Groq Whisper:
- `https://api.groq.com/openai/v1/audio/transcriptions`
- `whisper-large-v3-turbo`, then `whisper-large-v3`
- key: `GROQ_API_KEY`

2. NVIDIA fallback:
- `https://integrate.api.nvidia.com/v1/audio/transcriptions`
- `openai/whisper-large-v3`, `nvidia/whisper-large-v3`, and `nvidia/parakeet-tdt-0.6b-v2`
- keys: `NVIDIA_API_KEY` or `NVIDIA_NIM_API_KEY`

NVIDIA Whisper was retained as requested; it is a fallback, not the primary STT route.

Text-to-speech

- Primary: browser Web Speech API in the WebView.
- Windows fallback: PowerShell `System.Speech.Synthesis.SpeechSynthesizer` through SAPI.
- Misclick warning: pre-generated WAV plus Windows `PlaySoundW`, not live API speech.

Local APIs and data

- Windows UI Automation: element names, bounds, control types, events, and tree snapshots.
- xcap: monitor/window enumeration and screenshots.
- Local usage tracker: answers time-spent questions without an AI call.
- Tauri IPC: typed TypeScript-to-Rust command bridge.

10. Environment configuration

Current variable names are:

```
GROQ_API_KEY=
GEMINI_API_KEY=
GOOGLE_API_KEY=
OPEN_ROUTER_API_KEY=
OPENROUTER_API_KEY=
NVIDIA_API_KEY=
NVIDIA_NIM_API_KEY=
```

Only put real values in `.env.local` or another ignored local environment file. The report intentionally does not include any secret values. Restart Pointy after changing the file because the Rust process reads the configuration at runtime.

11. Privacy and security behavior

- No screenshot capture while idle.
- No continuous video/frame streaming.
- A screenshot is taken when the user asks a question and after a locally verified guided completion.
- The screenshot is cropped to the selected window when direct capture is available.
- API keys stay in the Rust process and are not sent to the webview.
- Usage questions are answered locally.
- Model output is parsed and scrubbed before display or speech.
- The application should still publish a formal retention policy before release. The current implementation does not constitute a complete security audit.

12. Current UI behavior

- App/window picker before asking.
- Voice or typed question input.
- Answer history with retry, copy, edit, and point controls.
- High-contrast target border, center dot, slow glow, and warning flash.
- Guided banner with one instruction at a time.
- Repeat that and Stop controls.
- No call-a-person panel.
- Speech mute/unmute control.
- Trust phase shown as a compact status label.

- Diagnostics are available through `guide://diagnostic` and Rust `TRUST:` logs.

13. Known limitations and honest status

- Windows is the verified platform. macOS Accessibility observer registration and coordinate Y-flipping are not complete.
- Accessibility quality depends on the target application exposing a useful UI Automation tree. When no element can be resolved, Pointy cannot safely start guided mode.
- The optional AI-suggested extra confusion zones are not implemented; current zones are geometric and accessibility-tree based.
- Free providers can return 429 responses, decommission models, or have network delays. The fallback cascade reduces but cannot eliminate that dependency.
- There is no local offline vision model in the current build.
- There is no billing, account system, wake-word engine, or full release installer workflow in the current implementation.
- The ignored live smoke tests require a real desktop window and live provider keys. Unit tests do not prove provider availability.
- A perfect experience cannot be guaranteed by the current free API setup; provider quotas and external model behavior remain outside Pointy's control.

14. Verification snapshot

The latest local verification completed before this report:

- Rust debug check: passed.
- Rust library tests: 34 passed, 6 intentionally ignored.
- Rust release check: passed.
- Rust release library tests: 34 passed, 6 intentionally ignored.
- TypeScript check: passed.
- Production frontend build: passed.
- No dev server was started for this report.
- No API key values are included in this report.

15. Useful commands after development resumes

From `pointy-software`:

```
pnpm run check
pnpm run build
pnpm tauri dev
```

From `pointy-software/src-tauri`:

```
cargo check
cargo test --lib
cargo check --release
cargo test --lib --release
```

Ignored live tests are intentionally separate because they open windows, capture the screen, use live providers, and may consume quota.

16. Bottom line

Pointy currently has a strong local control loop around an imperfect cloud vision suggestion:

explicit question
-> selected-window screenshot
-> free-provider vision model
-> structured answer and approximate label
-> Windows UI Automation refinement
-> physical center dot
-> local verification for guided progress
-> next screenshot/model call only after verified progress

The most important quality property is now explicit: Pointy should not silently assume that a user completed a step. The remaining work should focus on real desktop testing, provider reliability, and platform coverage rather than adding more speculative features.